Arush Singh

SMART CONTRACT & ZK DEVELOPER

github.com/04arush • in/04arush • x.com/04arush
04arush@gmail.com • New Delhi, India

PROFILE

Blockchain developer specialising in **Solidity Smart Contract** development and **Zero-Knowledge Cryptography**, with a focus on **DeFi Protocols**, on-chain automation, and rigorous testing. Built and deployed production contracts integrating **Chainlink VRF & Automation**, implemented constant-product AMM mechanics, and authored ZK circuits in **Noir** with on-chain proof verification via **Barretenberg UltraHonk**. Grounded in smart contract security principles — reentrancy, access control, invariant & fuzz testing — and actively advancing toward a **ZK Engineer** specialisation.

SKILLS

SMART CONTRACTS

Solidity | Foundry | DeFi | Chainlink VRF / Automation / Oracle | OpenZeppelin | Sepolia

SECURITY & TESTING

Fuzz / Unit Testing | Invariant Testing | ReentrancyGuard | Access Control | SC Security Basics

ZK / CRYPTOGRAPHY

Noir | Circom | Barretenberg (bb) | UltraHonk Verifier | SNARKs / STARKs | ZK Rollups

TOOLING & LANGUAGES

Rust | Linux | Git | Forge / Cast / Anvil | Etherscan

PROJECTS

ZK-Powered Undercollateralized Lending Prot.

↗ github.com/04arush/ZK-Powered-Uncollateralized-Lending-Pool

Noir · Barretenberg · Solidity · Foundry · OpenZeppelin · Sepolia

A DeFi lending protocol that enables undercollateralised borrowing (50% collateral ratio vs the industry-standard 150%) by verifying off-chain creditworthiness with a ZK proof — without revealing the user's credit score or income on-chain. The Noir circuit takes `credit_score` and `income` as private inputs and asserts both exceed protocol-defined public thresholds; `nargo prove` produces a proof which is passed directly to `LendingPool.borrow()`, where the on-chain `UltraVerifier` (generated by Barretenberg) validates it before any funds move. Core contract implements `deposit`, `borrow`, `repay`, and `liquidate` with strict Solidity conventions, NatSpec, `ReentrancyGuard`, and `SafeERC20`. Test suite achieves **100% line, statement, branch, and function coverage** across 23 passing tests: unit tests covering all happy and revert paths, fuzz tests (256 runs each) for random amounts and multi-user independence, and invariant tests running 256 sequences × 500 calls (128,000 actions per invariant) enforcing solvency, accounting integrity, and collateral floors via ghost-variable tracking. Deployed to Sepolia across 5 iterations; ZK proof generation runs entirely in-browser via Barretenberg WASM so private data never leaves the user's device.

Decentralized Provably Fair Raffle

↗ github.com/04arush/Raffle-Contest

Solidity · Chainlink VRF v2.5 · Chainlink Automation · Foundry

A fully autonomous on-chain raffle that requires zero admin intervention from entry through to prize payout. Randomness is sourced from **Chainlink VRF v2.5** — a subscription-based, cryptographically verifiable RNG — ensuring no party can predict or manipulate the winner. **Chainlink Automation** monitors `checkUpkeep` (time elapsed, raffle open, non-zero balance, participants present) and autonomously triggers `performUpkeep` when all conditions are met. A `HelperConfig` pattern provides chain-aware deployment: on Anvil it spins up a `VRFCoordinatorV2_5Mock` and a mock LINK token; on Sepolia it wires to live Chainlink contracts. The deploy script auto-handles subscription creation, LINK funding, and consumer registration in a single broadcast. 12 unit tests cover all state transitions, event emissions, access control, and upkeep logic; a fuzz test (256 runs) validates `fulfillRandomWords` can only be called after `performUpkeep`; a `skipFork` modifier gates VRF-mock tests to Anvil only.

AMM Decentralized Exchange (DEX)

↗ github.com/04arush/AMM-DEX

Solidity · OpenZeppelin · ERC-20 · Foundry · Sepolia

A constant-product automated market maker (x·y=k) supporting swaps between two custom ERC-20 tokens (AgoutiHusk / Utonagan) with liquidity provision, fee accrual, and slippage protection. Liquidity providers mint LP tokens proportional to their share of the pool (geometric mean for first deposit, pro-rata thereafter) and burn them to withdraw. Swap output is calculated with a 0.3% fee applied to the input amount before the constant-product formula, ensuring the invariant can only increase. Reentrancy protection via `ReentrancyGuard` guards all state-changing functions. 10 tests pass including fuzz tests verifying the x·y=k invariant holds post-swap, LP tokens scale proportionally with deposit size, and the pool can never be fully drained for any valid input amount. Deployed to Sepolia with initial liquidity seeded by the deploy script.

HACKATHONS

Frostbyte Hackathon — Finale

↗ frostbyte-hackathon.devpost.com

Devpost · Online · Results Pending

Advanced to the finale of the Frostbyte Hackathon series after placing in the qualifier round. Built the **ZK-Powered Undercollateralized Lending Protocol** — a Noir + Barretenberg + Solidity stack enabling privacy-preserving credit verification on-chain. Finalist results pending announcement.

Frostbyte Hackathon — Qualifier

↗ frostbyte.devpost.com

Devpost · Online · Participation & Completion Recognition

Submitted the **Employee Payroll Manager** — a Chainlink Automation–powered on-chain payroll system — earning Participation & Completion Recognition and an invitation to the Frostbyte Finale.

EDUCATION

Indira Gandhi National Open University (IGNOU)

Aug 2023 - Ongoing

Bachelor's — Computer Application

CERTIFICATIONS

Noir Programming & ZK Circuits	Cyfrin Updraft · Apr 2026
Fundamentals of Zero-Knowledge Proofs	Cyfrin Updraft · Mar 2026
Advanced Foundry	Cyfrin Updraft · Mar 2026
Foundry Fundamentals	Cyfrin Updraft · Mar 2026
Solidity Smart Contract Development	Cyfrin Updraft · Jan 2026
Rust Programming Basics	Cyfrin Updraft · Feb 2026
Advanced Web3 Wallet Security	Cyfrin Updraft · Feb 2026
Web3 Wallet Security Basics	Cyfrin Updraft · Feb 2026
Blockchain Basics	Cyfrin Updraft · Dec 2025